NAVER CONNECT

boostcamp ai tech

GENERATION FOR NLP 프로젝트
수능 문제 풀이 모델 생성

팀 모둠초밥

강감찬

단이열

안혜준

유선우

유채은

이채호

1. 프로젝트 개요

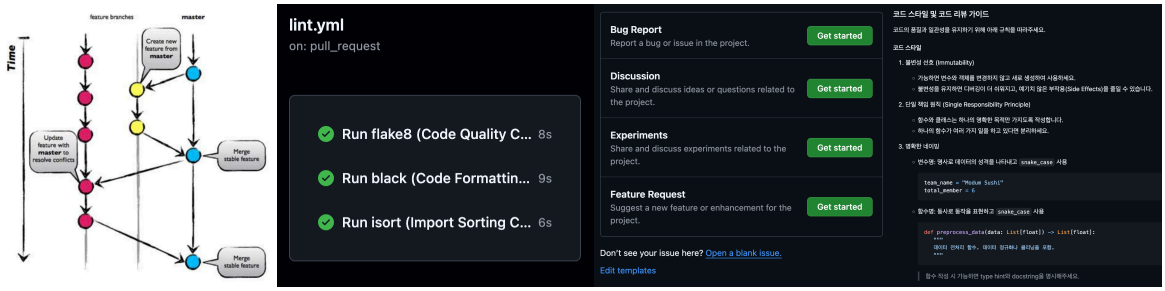
A. 프로젝트 목표

이번 프로젝트의 목표는 작은 규모의 언어 모델을 수능 문제 풀이에 최적화하는 것이다. 한국어의 특성과 수능 시험의 특징을 활용하며, 모델의 성능을 극대화하기 위해 다양한 접근법과 최적화 방안을 시도하였다.

B. 팀 구성 및 역할

팀원	역할
강감찬	Streamlit app 개발 참여, 데이터 관찰 및 분석, 데이터 재구성 및 증강
단이열	Streamlit app 개발 참여, RAG 구현 및 성능 평가
안혜준	협업 초기 환경 세팅 및 코드 모듈화, CoT 방식 실험 설계 및 성능 평가
유선우	베이스라인 모듈화, 메모리 사용 최적화, 모델 서치 및 실험
유채은	Streamlit app 개발 참여, 데이터 분석 및 정제, RAG 구현 및 성능 평가
이채호	문서 작업, RAG 전용 Vector DB 구성, 벤치마크 데이터셋 기반 데이터 증강

C. 프로젝트 환경



- **브랜치 관리 전략** : Github Flow를 채택하여 main, feature, hotfix 브랜치를 사용하기로 하고, 각 브랜치의 역할을 명확히 정의했다. 이를 통해 작업 간 충돌을 최소화하고, 안정적이고 효율적인 코드 병합 프로세스를 구축했다.
- **포맷팅 통일** : Black, isort, flake8을 활용해 코드 포맷팅을 표준화함으로써, 팀원 간 코드 스타일 차이에 의한 불필요한 수정 내역을 줄였다. 또한, Github Actions를 통해 새로 올라온 PR의 포맷팅 상태를 자동으로 검토하여 작업의 일관성을 확보했다.
- **이슈 및 PR 관리** : 체계적인 라벨링과 Issue/PR 템플릿을 도입하여 작업의 우선순위와 상태를 명확히 관리했다. 이를 통해 팀원 간 작업 흐름을 투명하게 공유하고, 개발 과정의 혼선을 최소화했다. 또한, 긴급한 작업이나 장기적인 과제를 한눈에 파악할 수 있도록 했다.
- **개발 문화 표준화** : CONTRIBUTING.md 파일을 통해 이슈 작성, 커밋 메시지 작성 규칙, 브랜치 네이밍 방식, 코드 스타일 가이드를 명시해 팀원들이 일관된 방식으로 협업할 수 있도록 했다.

2. 프로젝트 수행 과정 및 결과

A. 초기 개발 세팅

1) 베이스 라인 코드 모듈화

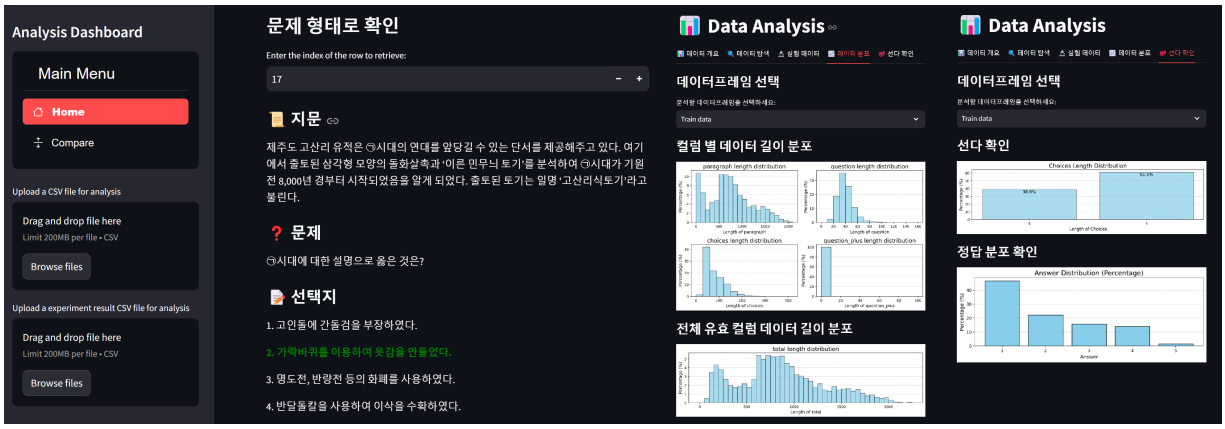
```
project/
├── configs/
│   └── config.yaml                # 실험 파라미터 설정 파일
├── data_loaders/
│   ├── base_data_loader.py
│   └── data_loader_factory.py     # DataLoader 생성 factory
├── prompts/                       # 프롬프트 템플릿 디렉토리
│   └── templates/
│       └── base/                  # 기본 프롬프트 템플릿
├── train.py
└── inference.py
```

실험의 일관성을 유지하고 코드 재사용성을 높이기 위해 프로젝트 구조를 모듈화하였다. config.yml 파일을 중심으로 실험 파라미터를 관리하여, 설정 변경 시 코드 수정 없이 손쉽게 실험 환경을 재구성할 수 있도록 설계했다.

데이터 로딩 과정은 Template Method Pattern을 적용하여 상위 클래스에서 공통 로직을 처리하고, 데이터 형식에 따라 하위 클래스에서 구현하도록 설계했으며, Factory Method를 통해 config 설정에 맞는 DataLoader 객체를 동적으로 생성할 수 있도록 하였다. 또한, prompts/templates 디렉토리에 텍스트 파일 형식으로 prompt 템플릿을 저장하고 config 파일로 관리함으로써 실험 환경에 맞게 유연하게 활용할 수 있도록 했다.

이러한 모듈화 설계는 다양한 데이터 및 환경 조건에서도 일관되고 효율적인 작업이 가능하도록 기여했다.

2) 데이터 관찰 및 분석용 도구 개발



Jupyter Notebook에서 데이터 관찰 과정 중 코드 수정 및 셀 실행을 반복하는 과정이 비효율적이라는 생각이 들었다. 이러한 불편함을 개선하기 위해 Streamlit을 도입하였다. 이를 통해 직관적이고 유연한 데이터 탐색 환경을 제공하며, 별도의 코드 작성 없이 데이터를 확인하고 시각화할 수 있도록 지원하였다. 또한, 데이터 구조 변경에도 유연하게 대응할 수 있는 일반화된 시스템을 구축하여 코드 중복을 줄이고 워크플로우를 단순화하며, 팀원 간 협업 효율성을 높였다.

3) 고정된 검증 데이터셋 설정

기존에는 학습 데이터에서 검증 데이터를 랜덤으로 분할하여 사용했으나, 검증 데이터의 신뢰성과 실험 결과의 재현성을 높이기 위해 400개의 고정된 검증 데이터셋(validation set)을 별도로 구성하기로 했다.
이 과정에서 검증 데이터와 학습 데이터의 분포가 유사하도록 설계하는데 중점을 두었다. 검증 데이터가 전체 데이터의 대표성을 가질 수 있도록, 토큰 길이를 기준으로 기존 학습 데이터의 분포와 최대한 일치하도록 검증 데이터를 분할했다. 이렇게 구성된 검증 데이터셋은 데이터 증강 및 필터링 과정에서도 신뢰성을 유지하며, 모델 평가의 일관성을 보장할 수 있다.
이를 검증하기 위해 400개 데이터를 검증용으로 제외하고 1600개의 데이터로 학습한 모델과, 2000개 데이터를 모두 학습한 모델의 테스트 점수를 비교했다.

모듈화 test 최종 2000개 학습		0.4194 0.3747	2024.11.19 15:01
validation 4...안겹치게) 1600개 학습		0.4124 0.3977	2024.11.20 15:14

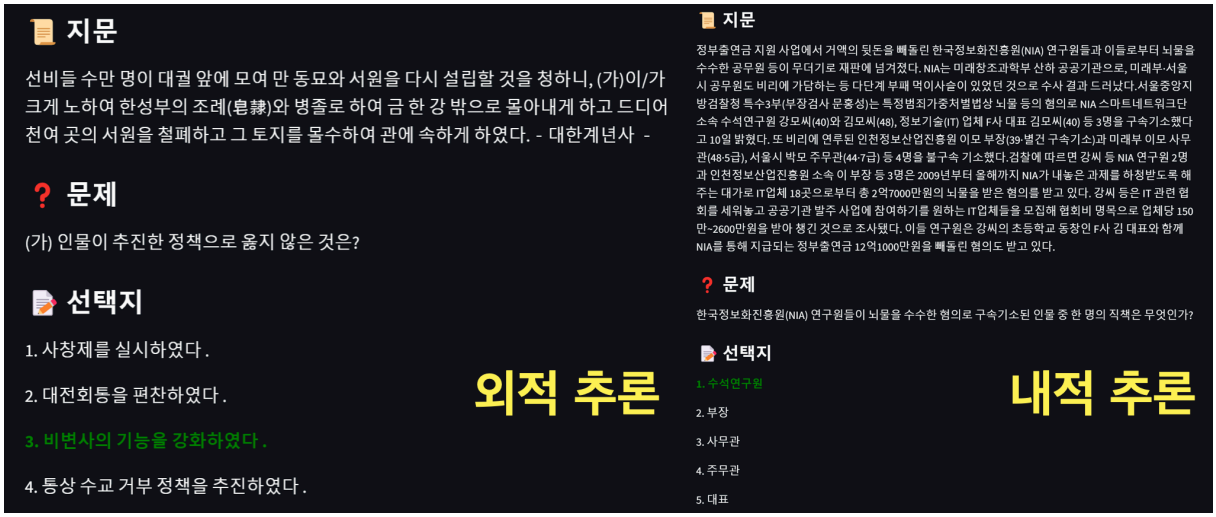
그 결과, 두 모델의 테스트 점수가 거의 유사하게 나타났으며, 학습 데이터와 검증 데이터가 기존 데이터 분포를 잘 반영하고 있음을 확인했다. 이러한 설정은 검증 데이터의 신뢰성을 유지함과 동시에, 실험 결과의 재현성과 성능 비교의 일관성을 높이는 데 기여했다.

B. 데이터 관찰 및 분석

프로젝트 초기 단계에서 Valid와 Test 점수 간 큰 차이가 관찰되었다. 이러한 차이는 Test 데이터 분포와 Train 데이터 분포 사이에 불일치가 존재할 가능성을 시사했다. Test 데이터의 분포를 직접 확인하는 것은 규칙에 어긋나기 때문에, Train 데이터의 구성과 특성을 면밀히 분석하는 방식으로 접근했다. Train 데이터는 수능형 문제와 MRC 데이터, KMMLU 데이터, 그리고 MMMLU 데이터로 구성되어 있었으며, 이를 상세히 분석하여 문제의 원인을 찾고자 했다. 우선, Train 데이터의 지문 길이를 분석한 결과, MRC, KMMLU, MMMLU 데이터를 기반으로 한 문제들의 지문 길이가 수능형 문제에 비해 현저히 짧았다.

영역	2021학년도	2022학년도
주제통합	2,238자	1,961자
사회	1,532자	1,429자
기술	1,618자	1,036자
독서이론	-	899자

실제 수능 국어 영역에서 평균 지문 길이를 확인했을 때, 대체적으로 1000자를 훨씬 상회하였다. 반면에 학습 데이터의 평균 지문 길이는 이보다 훨씬 짧은 735자로, 수능 문제 스타일과는 분명한 차이를 보였다. 이는 모델이 학습하는 과정에서 충분한 맥락 정보를 제공받지 못할 가능성을 시사하며, 논리적 추론 과정을 학습하는 데 제한이 있을 수 있다고 생각했다.
이러한 관찰을 바탕으로 외적 추론과 내적 추론이라는 개념을 도입하여 문제를 분류하기로 했다. 외적 추론은 문제 해결을 위해 외부 지식이나 배경 정보를 필요로 하는 유형으로, 지문과 문제 내에서 제공된 정보만으로는 정답에 도달하기 어려운 경우를 포함한다. 반면, 내적 추론은 지문에서 제시된 정보와 논리적 연결을 통해 문제를 해결할 수 있는 유형으로, 텍스트 자체의 맥락과 논리에 기반한 사고를 요구한다.



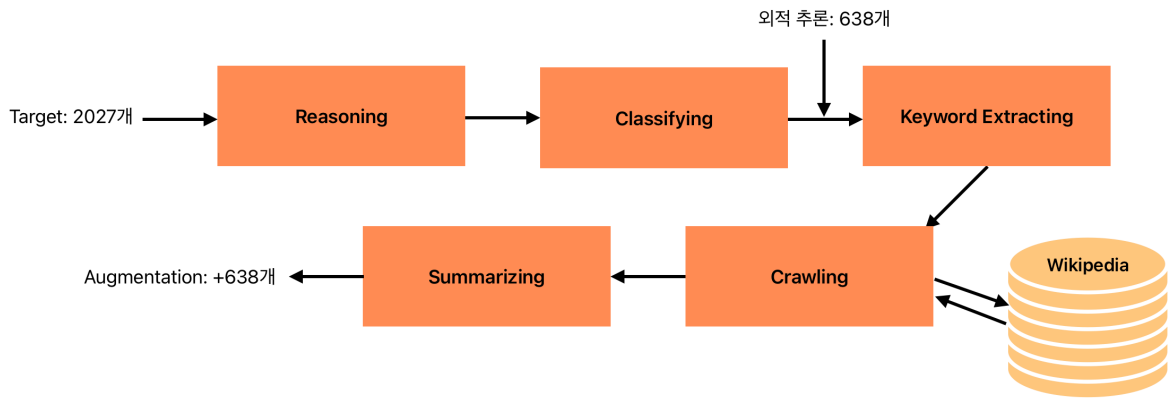
이와 같은 분류 기준을 도입한 이유는 수능 문제의 본질적 특성을 반영하기 위한 것이다. 수능은 지문에서 제공된 정보를 논리적으로 분석하고 추론하는 능력을 평가하는 데 중점을 둔다. 하지만 기존 학습 데이터셋은 외부 지식의 활용을 요구하는 경우가 많아 논리적 사고와 맥락적 추론을 강화하는 데 한계가 있다고 느꼈다. 따라서 외부 지식이 필요한 외적 추론 문제에 지문을 보강하여, 내적 추론 문제로 전환하는 방법을 떠올렸고, 이렇게 재구성 또는 증강된 데이터가 “수능에 최적화된 모델” 학습에 적합할 것이라 기대했다.
데이터를 재구성하고 증강하는 과정에서, 특정 문제가 외적 추론 문제인지 내적 추론 문제인지 판단하기 위해 GPT API를 활용하였다. 두 분류를 구분하는 기준을 수립한 뒤, GPT API에게 문제 풀이 과정을 분석하여 외부 지식이 필요한 문제인지 여부를 판단하도록 하는 실험을 진행했다.

C. 데이터 재구성 및 증강

1) 외적 추론 문제에 대한 재구성 및 증강

외적 추론 문제로 분류된 데이터를 내적 추론 문제로 변환하기 위해 다음과 같은 절차를 수행하였다.

- 핵심 키워드 추출: 문제 해결에 필요한 사건, 인물, 단체 등 주요 정보를 기반으로 핵심 키워드를 도출했다.
- 관련 지문 확보: 추출된 키워드를 바탕으로 한국어 위키피디아에서 관련 문서를 크롤링하여 문제 풀이에 필요한 정보를 수집하였다.
- 문제 지문 보강: 확보된 문서를 문제 지문에 추가하여, 지문 내 정보만으로 문제를 풀 수 있는 형태로 변환했다. 이 때, 크롤링된 문서가 지나치게 방대한 경우, 모델의 입력 시퀀스 길이를 고려해 적절한 길이의 요약문을 생성해 대체하였다.



베이스라인/gemma-...b/테스트	증강 전	0.3940 0.3862	2024.11.21 22:16
베이스라인/기본데이터(...찬증강추가	증강 후	0.3710 0.3862	2024.11.27 20:59

모델 학습에 증강된 데이터를 활용한 결과, 성능 향상에는 실패하였으며 오히려 성능 저하가 발생했다. 이는 일부 수능형 문제조차 외부 지식에 크게 의존한다는 점을 보여주었다. 이 발견은 단순히 학습 데이터의 재구성을 넘어, 외부 지식을 효과적으로 활용할 수 있는 방법론의 필요성을 제기했다. 이에 따라, 프로젝트에서는 이후 RAG(정보 검색 기반 생성) 접근법의 도입과 활용 가능성을 탐구하는 방향으로 이어지게 되었으며, 이는 D 파트에서 상세히 다룰 예정이다.

2) 벤치마크 데이터셋을 통한 데이터 증강

비수능형 문제는 대부분 외적 추론 문제일거라는 나이브한 가정도 문제였지만, GPT를 활용하여 증강한 데이터의 품질에도 문제가 있다고 판단되었다. GPT로 생성된 데이터는 문항의 문맥적 일관성과 Test 데이터와의 연관성이 부족하여 모델 성능을 개선하기는커녕 오히려 저하시키는 결과를 초래했다.

Korean Benchmarks	Translated	Public?	# Category	# Instances	Type
KorNLI & KorSTS	O	O	2	6,389	Understanding
KLUE	X	X	8	38,500	Understanding
Ko-H5	O	X	5	25,700	Reasoning
KoBBQ	△	O	268	76,048	Bias
HAE-RAE Bench.	X	O	6	1,538	Cultural Knowledge
CLiCK	X	O	11	1,995	Cultural Knowledge
KorNAT	X	△	16	10,000	Cultural Alignment
KMMLU (Ours)	X	O	45	35,030	Expert Know. & Reasoning

이러한 문제를 해결하기 위해 데이터 품질의 중요성을 인식하고, 기존 학습 데이터를 보강할 수 있는 벤치마크 데이터셋을 활용하는 방안을 검토하였다.

높은 품질의 데이터를 확보하기 위해 KMMLU 벤치마크의 논문에서 언급된 벤치마크 중, Public으로 공개된 벤치마크 데이터셋 (KOR NLI & KOR STS, KoBBQ, HAE-RAE, CLICK) 을 분석하였다. 그 결과, KorNLI와 KorSTS는 다지선다형으로 가공하기 어려웠고, KoBBQ는 모델 학습 방향과 직접적인 연관성이 부족해 최종적으로 제외되었다. HAE-RAE와 CLICK 데이터셋이 유력한 후보로 남았으며, HAE-RAE 데이터셋의 경우 다지선다형 데이터로 가공 가능하고 학습 목표와 부합하는 13개 카테고리 중 <낱말 정의 매칭, 상식, 한국사, 희귀어, 독해, 전문용어> 6개 카테고리를 선별하였다.

이후, HAE-RAE 데이터셋을 활용한 다양한 조합을 실험하였다. 먼저 6개 카테고리를 모두 추가했을 때는 데이터 간 중복성과 Test 데이터와의 낮은 연관성으로 인해 성능이 소폭 상승(0.3986 → 0.4055)에 그쳤다. 독해 데이터셋과 상식을 조합했을 때는 상식 카테고리의 Question Plus 항목을 활용했음에도 불구하고 성능 상승폭이 감소하여 0.4009에 머물렀다. 낱말 정의 매칭 데이터를 독해와 조합하여 Question의 다양성을 높였을 때는 0.4171로 성능이 일부 개선되었으나, 독해 데이터셋 단독 성능(0.4309)을 넘지 못했다.

베이스라인 코드 + H...이터 추가		0.4309 0.4138	2024.11.26 22:00

가장 성공적인 조합은 독해 데이터셋을 단독으로 추가한 경우였다. 이때 모델의 성능은 0.3986에서 0.4309로 대폭 상승했다. 이는 독해 데이터셋이 Test 데이터와 문제 형태에서 높은 연관성을 보였기 때문에 모델이 Test 데이터를 효과적으로 일반화할 수 있었던 결과로 분석된다.

결국, 데이터 증강에서 중요한 것은 단순히 데이터의 양이 아니라 데이터의 품질과 Test 데이터와의 연관성이라는 점을 다시 한번 확인하게 되었다. 기존의 단순한 데이터 생성 접근법은 성능 개선에 한계를 보였으며, 높은 품질의 데이터셋을 선별하고 이를 학습 데이터에 적절히 조합하는 것이 더 효과적임을 알 수 있었다. 앞으로는 데이터 품질을 더욱 엄격히 검토하고, Test 데이터와의 연관성을 정량적으로 분석하는 방향으로 접근할 필요가 있음을 느꼈다.

D. Retrieval-Augmented Generation (RAG)

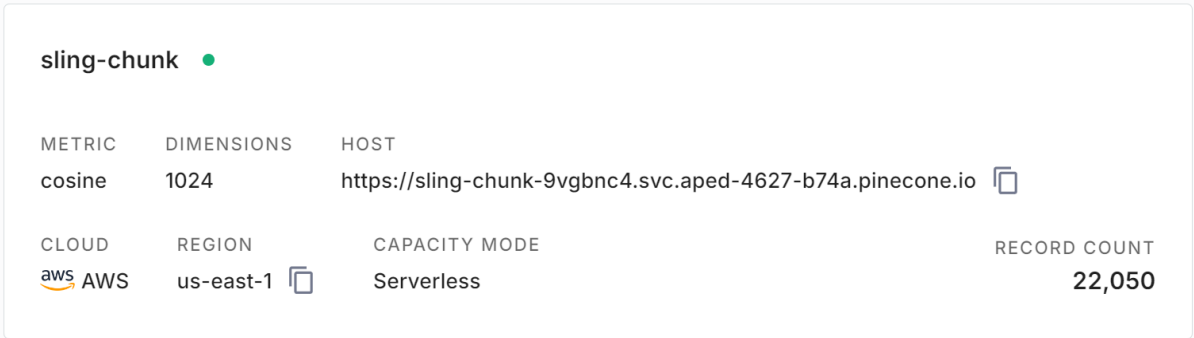
1) 도입 배경

앞서 내/외적 추론으로 데이터를 재구성하고 증강한 방식으로 모델을 학습하였을 때, 오히려 일부 경우 성능 저하를 보였던 문제를 확인했다. 이는 단순한 데이터 재구성의 한계를 넘어, 복잡한 문제 해결에 필요한 외부 지식을 효율적으로 통합할 방법론의 부재를 드러낸다. 특히, 일부 수능형 문제에서도 외부 정보에 크게 의존하는 양상이 나타났다는 점은, 보다 정교한 접근법이 필요하다는 신호로 볼 수 있다.

이에 프로젝트는 기존의 단순한 학습 데이터 기반 접근을 넘어, 외부 지식을 효과적으로 검색하고 이를 기반으로 텍스트를 생성할 수 있는 **정보 검색 기반 생성(Retrieval-Augmented Generation, RAG)** 방법론의 도입을 모색했다. 이 접근법은 모델의 한계를 극복하고, 정보 의존적인 문제 해결 능력을 높일 가능성을 제공한다. 아래 섹션에서는 RAG의 핵심 단계와 이를 기반으로 한 실험 및 분석 결과를 상세히 다룰 것이다.

2) 구성 요소 설명

- **Vector DB :** 대규모 데이터의 검색 효율성을 위해 **Pinecone**을 선택하였다. Pinecone은 클라우드 기반으로 설계되어 데이터 규모의 확장에 유연하며, 낮은 지연 시간으로 실시간 검색이 가능하다는 장점이 있다. 또한, Python API를 활용한 쉬운 통합으로 개발 및 실험 속도를 높일 수 있었다. Pinecone의 성능은 특히 다수의 청크 데이터에서 효율적인 검색과 인덱싱을 보장하며, RAG의 Retrieval 과정에서 핵심 역할을 수행하였다.
- **데이터 구성** : 데이터는 주로 위키피디아 문서를 기반으로 구축되었다. 그러나 단순히 모든 위키피디아 문서를 수집하는 방식은 수능과 무관한 문서들이 포함될 가능성이 있다고 판단되었다. 이를 최소화하기 위해, 이전 실험에서 추출된 키워드들을 활용하여 이와 관련된 위키피디아 문서들만 선별적으로 크롤링하였다. 또한, 초기 크롤링 과정에서 문서의 각주, 링크 등 불필요한 부분이 Retrieval 과정에 포함되는 문제가 발견되었다. 이를 해결하기 위해 크롤링한 위키피디아 문서를 정제하여 중요한 본문 정보만 남긴 정제된 데이터를 Pinecone에 재구축하였고, 최종적으로 22,050개의 임베딩 벡터를 가지는 Vector DB를 구성하게 되었다.



- **임베딩 모델 선정** : 프로젝트에서 선택한 벡터 임베딩 모델은 **bge-m3**로, 이 모델은 다양한 텍스트와 언어에서 높은 표현력을 보여준다. bge-m3는 문장과 단락의 문맥을 깊이 있게 반영하는 임베딩을 생성함으로써, 단순한 키워드 매칭 이상의 고차원적인 유사성을 측정할 수 있는 강점을 갖고 있다. 다국어 텍스트와 다양한 도메인을 다루는 작업에서도 우수한 성능을 발휘하며, 프로젝트 데이터셋처럼 복잡한 문장 구조와 특정 도메인에 특화된 학습 데이터와도 잘 적응한다. 더불어, bge-m3는 경량화된 구조를 통해 대

규모 데이터 처리에서도 상대적으로 낮은 계산 자원을 요구한다. 이는 Pinecone과 연계하여 실시간 검색을 수행하는 과정에서 중요한 장점으로 작용했다.

- **Retrieval 방식** : RAG 구성에서 Retrieval 단계는 검색된 외부 데이터의 정확성을 높이는 핵심 과정으로, 이 프로젝트에서는 **Cosine Similarity Search**를 채택하였다. 이 방식을 선택한 이유 중 하나는 검색 대상 데이터의 지문(paragraph) 길이가 다소 길거나 짧은 경우가 혼재되어 있었기 때문이다. 이로 인해 벡터 임베딩 후 크기가 제각각이 되는 문제가 발생할 수 있었는데, Cosine Similarity는 두 벡터 간 각도를 기준으로 유사성을 평가하므로 벡터 크기(길이)가 유사도 계산에 영향을 미치지 않는다는 점에서 DP(Dot Product)보다 적합하다고 판단했다.

3) 실험 결과 및 분석

실험 결과와 분석을 통해 RAG의 성능에 영향을 미치는 다양한 요인과 문제점을 확인하였다. 초기 테스트에서는 문제의 단락(paragraph)을 그대로 쿼리로 사용하여 RAG를 적용하였는데, validation 데이터에서는 성능이 상승했으나 test 데이터에서는 오히려 성능이 하락하는 현상이 나타났다.
이를 통해 쿼리 품질에 문제가 있을 가능성을 제기하였으며, 수능 문제의 특성상 문제에서 주어진 정보만으로는 정답을 도출하기 어려운 경우가 많아 추가 정보를 retrieval 단계에서 확보하기 어렵다는 점이 한계로 작용했음을 파악하였다.
쿼리 품질을 개선하기 위해 query builder를 개발하여 paragraph 외 다양한 조합의 쿼리를 테스트하였으나, 오히려 성능이 하락하는 결과가 나타났다. 이를 통해 단순히 쿼리 구성 방식의 문제가 아니라 다른 요인이 작용하고 있음을 감지하고, retrieve된 문서를 직접 검토하며 원인을 분석하였다. 이 과정에서 retrieve된 문서가 문제 풀이에 혼란을 줄 수 있는 정보를 포함하고 있는 경우가 발견되었으며, 이러한 문제를 체계적으로 분석하기 위해 LLM as Judge를 도입하게 되었다.

```
당신은 유능한 AI 평가위원입니다.
당신이 할 일은 주어진 문제 데이터(# Question Data)를 푸는데 지원 문서(# Support)가 얼마나 도움이 되는지 평가하는 것입니다.
문제 데이터(# Question Data)는 지문, 문제, 선택지, 정답으로 구성되어 있습니다.
평가 기준(# Evaluation Criteria)은 다음의 3가지 입니다.

# Evaluation Criteria
1. Support 문서에 문제 풀이에 **필요한 정보**가 전부 포함되었는가?(1점)
정의: 문제를 해결하기 위해 필수적으로 알아야 하는 정보가 문서에 서술되어 있는 경우 점수를 부여합니다.
점수를 받는 경우 Support를 활용하여 문제의 정답을 맞출 수 있습니다.

2. Support 문서에 문제 풀이에 **필요한 정보**가 일부 포함되었는가?(1점)
정의: 문제를 해결하기 위해 필수적으로 알아야 하는 정보가 문서에 서술되어 있는 경우 점수를 부여합니다.
점수를 받는 경우 Support를 활용하여 문제의 정답을 맞출 순 없지만, Support의 존재가 문제 풀이에 도움이 됩니다.

3. 문제 풀이에 **필요한 정보**가 Support 문서에서 핵심적으로 다루는 내용인가? (1점)
정의:  문제 풀이에 **필요한 정보**가 Support 문서의 주제나 목적과 밀접하게 관련이 있는 경우 점수를 부여합니다.

평가는 FORMAT에 맞춰 작성하세요.
```

Valid 데이터에서 랜덤으로 100개 샘플을 선택한 뒤, 각 샘플마다 top-k=3으로 retrieve한 3개의 문서를 concat하여 하나의 문서로 만들었다. 이렇게 생성된 총 100개의 문서를 대상으로, GPT API를 활용한 LLM as Judge 평가를 진행했다. 평가는 각 문서를 프롬프트에 명시된 기준에 따라 수행했다.

query	paragraph	paragraph + question	paragraph + question + choices	tf-idf
1. 전부	3	6	9	5
2. 일부	13	17	18	12
3. 핵심	1	2	4	1
총점	17	25	31	18

Retrieve 된 문서는 문제 해결에 필요한 정보를 포함하고 있는 정도에 따라 점수를 부여하는 방식으로 평가되었고, 결과적으로 각 쿼리 방식에 따른 valid 점수 순서와 유사한 경향을 보였다. 이를 통해 완벽하지는 않지만, LLM as Judge를 활용해 retrieve된 문서의 품질을 일정 부분 평가할 수 있음을 확인할 수 있었다.

4) 결론 및 논의

이번 실험은 단순히 성능 결과를 확인하는 데 그치지 않고, 성능 하락의 이유를 분석하며 특정 지표를 통해 평가하는 과정 자체가 의미 있었다고 느낀다.

rag prompt 수...opk=5	정제 전		0.5161 0.5103	2024.11.26 14:02
rag_baseline...opk=5	정제 후		0.5230 0.5172	2024.11.26 18:06

추가로, 데이터 정제 이후 Pinecone을 재구성했을 때 성능이 소폭 향상된 점을 통해, retrieve되는 문서의 품질이 RAG의 성능에 매우 중요한 영향을 미친다는 점을 다시 한번 확인할 수 있었다. 이러한 결과는 향후 RAG를 활용한 시스템 설계 및 개선 시, 데이터 정제와 문서 품질 관리에 더욱 신경 써야 함을 시사한다.

rag_baseline...blank	RAG 적용 X		0.5253 0.4943	2024.11.26 15:23

초기 test public 점수에서는 성능이 하락하여 RAG의 효용성에 의문이 들었으나, 최종적으로 공개된 test final 점수를 통해 RAG를 활용한 성능 향상이 있었음을 확인할 수 있었다. 이를 통해 RAG의 적용이 궁극적으로 시스템 성능 개선에 기여함을 알 수 있었다.

E. Chain of Thought (CoT)

1) 도입 배경

이번 프로젝트의 목표는 수능형 문제를 정확히 풀어낼 수 있는 모델을 개발하는 데 있다. 수능형 문제는 단순한 정답 선택을 넘어, 문제의 논리적 구조를 이해하고 단계적인 사고 과정을 요구하는 경우가 많다. 기존 모델들은 대개 단순히 정답 번호만을 출력하거나, 문제를 직관적으로 처리하는 방식에 의존했으나, 이러한 방식은 문제의 복잡한 추론 과정을 충분히 반영하지 못한다고 생각했다.

이에 따라, **Chain of Thought(CoT)** 방식을 실험적으로 도입했다. CoT는 사람이 문제를 푸는 과정처럼 단계별로 사고의 흐름을 모사하며, 복잡한 추론이 필요한 문제에서 특히 강점을 보인다는 연구 결과들이 있었다. 이 방식을 도입하면 모델이 단순히 정답만 도출하는 데 그치지 않고, 그 과정에서 필요한 논리적 단계를 더 명확히 이해하고 표현할 수 있을 것이라 기대했다.

2) 실험 과정

이번 실험에서는 두 가지 모델인 **google/gemma-2-2b-it**와 **Qwen/Qwen-2.5-7B-Instruct**를 대상으로 Chain of Thought(CoT) 방식을 적용하여 성능을 비교했다. 실험은 두 단계로 나뉘어 진행되었다. 첫째, 모델이 숫자(정답 번호)만 출력하도록 설계한 기본 방식과, 둘째, JSON 형식으로 reasoning을 포함해 답변을 생성하도록 설계한 CoT 방식을 각각 평가했다.

각 모델은 사전 학습(pre-trained) 상태와 추가 파인튜닝(fine-tuning) 상태에서 테스트를 진행했으며, 성능 비교를 위해 **테스트 데이터 800개**를 활용했다. 테스트 데이터는 주로 수능형 문제로 구성되었으며, 언어 및 사회 과목의 문제들이 주를 이루었다. 각 문제에 대해 모델이 도출한 답안과 reasoning의 정확도를 평가하여 점수를 부여했다.

google/gemma-2-2b-it 모델은 CoT 방식을 적용한 경우, 파인튜닝 모델에서 기존 숫자 출력 방식보다 성능이 감소했다. 이는 JSON 형식의 응답 생성이 모델에 과도한 복잡성을 요구했기 때문으로 보인다.

실험 설정	사전 학습 모델 (점수)	파인튜닝 모델 (점수)
숫자(정답 번호)만 대답하도록 모델 설계	0.5034	0.5264
JSON 형식의 reasoning 및 정답을 생성(CoT)	0.4885	0.4437

google/gemma-2-2b-it

반면 Qwen/Qwen-2.5-7B-Instruct 모델에서는 CoT 방식이 사전 학습 및 파인튜닝 상태 모두에서 성능을 향상시켰지만, 여전히 숫자만 대답하는 기본 설계의 성능을 넘어서지는 못했다.

실험 설정	사전 학습 모델 (점수)	파인튜닝 모델 (점수)
숫자(정답 번호)만 대답하도록 모델 설계	0.6092	0.6207
JSON 형식의 reasoning 및 정답을 생성(CoT)	0.5609	0.6092

Qwen/Qwen-2.5 7b Instruct

또한 JSON 응답 형식의 문제도 확인되었다. JSON 형식 응답은 기존 숫자 대답 방식에 비해 높은 정확성과 논리적 구조를 요구했지만, 약 800개 중 30개의 경우에서 모델이 JSON 형식을 제대로 생성하지 못하는 사례가 발생했다. 이는 JSON 응답 생성의 높은 복잡도와 CoT 방식의 처리 한계로 인한 것으로 판단된다.

3) 결론 및 논의

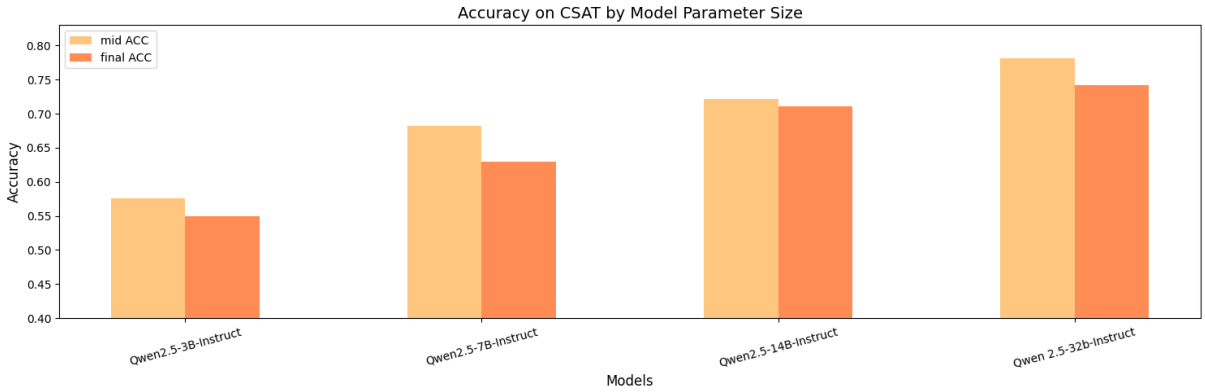
결론적으로, CoT 방식은 특정 유형의 문제, 특히 수학처럼 명확한 추론 단계가 필요한 문제에서 더 큰 효과를 발휘할 가능성이 있다. CoT 방식이 모델의 reasoning 성능을 보완할 잠재력을 보여준 만큼, 이를 기존의 RAG(Relational Answer Generation) 파트와 연계하여 활용하는 방안도 고려할 가치가 있다. 특히, RAFT(Retrieval-Augmented Fine-Tuning)가 기본적으로 CoT 스타일의 답변을 가정하고 설계된 만큼, 이번에 개별적으로 진행된 CoT 실험과 RAG 파트를 결합해 RAFT를 제대로 시도해보는 것도 유의미한 방향이 될 것이다. 이러한 통합적인 접근은 모델의 추론 능력을 보다 효과적으로 강화하는 데 기여할 수 있을 것으로 기대된다.

F. 대형 모델 양자화 및 성능 최적화

1) 대형 모델 도입 배경

최근 대규모 언어 모델(LLM)의 성능 평가에서 MMLU 벤치마크는 언어 모델의 이해 능력을 측정하는 대표적인 기준으로 자리 잡았다. 그러나 이 벤치마크는 언어 생성 능력과는 거리가 있어, 실제 모델의 활용 가능성을 온전히 반영하지 못한다는 한계가 있다. 예를 들어, 한국어 오픈소스 리더보드에서 상위권에 있는 일부 모델은 언어 이해에서는 뛰어난 성능을 보이지만, 언어 생성에서는 부족한 결과를 보인다. 이는 **이해 능력과 생성 능력이 별개의 영역**임을 보여준다.

이해 능력은 모델이 데이터를 암기한 수준에 크게 의존하며, 암기 능력은 모델의 크기와 직접적으로 연결된다. 특히 대규모 모델은 더 많은 데이터를 저장하고 활용할 수 있어 이해 중심의 벤치마크에서 더 높은 점수를 얻는다. 이번 프로젝트의 목적이 명목상으로는 생성형 모델을 만드는 것이지만, 실질적으로는 1부터 5의 선택지 중에서 정답을 고르는 분류 모델을 학습하는 것이다. 그렇기에 생성 능력보다는 이해 능력이 더 중요할 수도 있겠다는 생각이 들었다.

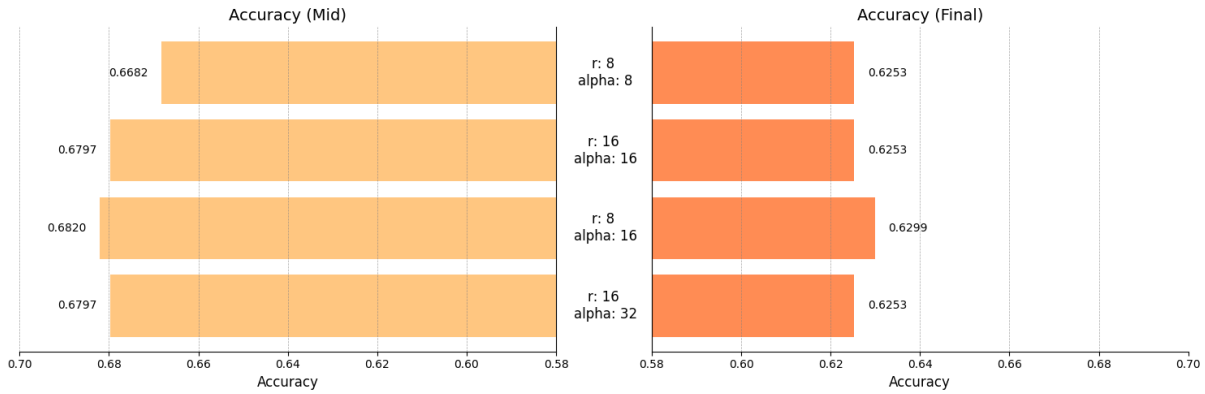


이와 같은 배경에서 우리는 **32B 크기의 Qwen 모델**을 활용하여 한국어 이해 능력을 최적화하고자 했다. 그러나 32B와 같은 대규모 모델을 활용하려면, GPU 메모리 제한이라는 현실적 문제를 극복해야 한다. 이를 해결하기 위해, 양자화와 다양한 메모리 최적화 기법을 적용하여 대규모 모델을 V100 GPU에 올리는 시도를 하였다.

2) 양자화 및 메모리 최적화

모델의 메모리 사용량을 줄이기 위해 가장 먼저 **8비트와 4비트 양자화**를 적용했다. Bitsandbytes 라이브러리를 사용하여 모델의 메모리 요구량을 낮추면서도 연산 효율성을 극대화할 수 있었다. 여기에 더해, **Unsloth** 라이브러리를 활용해 메모리 사용량을 줄이고 학습 속도를 향상시켰다.

또 다른 메모리 최적화 방법으로는 **Gradient Checkpointing**을 적용했다. 이 기법은 학습 중 순전파 단계에서 활성화 값을 저장하지 않고, 역전파 시 필요할 때 다시 계산하는 방식으로 메모리 사용량을 획기적으로 줄여준다. 또한 **Gradient Accumulation** 기법을 활용하여 작은 배치 크기로 여러 번 순전파를 수행한 후 역전파를 한 번에 진행함으로써 메모리 사용량을 더욱 줄일 수 있었다.



이외에도 **LoRA**를 적용하여 미세조정 단계에서 필요한 파라미터 수를 줄였다. LoRA는 특정 파라미터(r과 alpha) 값을 실험적으로 조합해 최적의 성능을 확인할 수 있었으며, 특히 r 값 8, alpha 값 16의 조합에서 가장 좋은 결과가 나타났다. 이와 함께 **torch_dtype을 float16으로 설정**하여 메모리 사용량을 추가로 줄이고 연산 속도를 높였다.

마지막으로 학습 세부 설정에서도 최적화를 시도했다. **Max Sequence Length**를 2048로 설정해 더 많은 토큰을 처리할 수 있도록 했으며, **2 Epoch**에서 가장 안정적인 학습 결과를 보임을 확인했다. 학습 중 불필요한 연산을 줄이기 위해 Early Stopping을 적용했으며, 학습률은 2e-5로 고정해 과적합을 방지하고 학습을 안정화했다.

3) 성능 및 결과 분석

Qwen2.5-7B/n...0/COT		7B	0.6014 0.6092	2024.11.26 10:13
Qwen2.5-32B-...1e-05		32B	0.7788 0.7540	2024.11.27 14:03

앞서 적용된 RAG나 CoT에서 성능 향상을 보기 어려웠던 것과 달리, 32B 모델을 활용한 이번 실험에서는 **비약적인 성능 향상**을 확인할 수 있었다. 특히, 생성 능력과 이해 능력을 동시에 요구하는 작업에서 대형 모델이 기존 소형 모델 대비 압도적인 성능 우위를 보여주었다. 이는 멘토링에서 제안된 "파라미터가 큰 모델일수록 이해 능력에서 더 높은 성능을 보인다"는 통찰과도 일치하며, 대형 모델의 암기와 이해 능력이 성능 향상에 중요한 역할을 한다는 점을 다시 한 번 입증한 결과다.

G. 프롬프트 엔지니어링

1) 프롬프트 설계

이번 실험에서는 **다양한 프롬프트 접근법**을 비교하여 성능을 극대화할 수 있는 최적의 템플릿을 선정하고자 했다. AI 수능 국어 만점 프로젝트(NomaDamas/KICE_slayer_AI_Korean)의 사례를 참고하여 진행되었으며, 이는 수능 국어 만점을 목표로 한 다양한 프롬프트 방식의 실험적 사례를 제공한다. 이를 기반으로 우리는 문제를 단계적으로 해결하거나 (Zero-shot CoT), 체계적인 지침을 제공하는 방식 (Plan and Solve), 또는 감정적 요소를 추가하여 모델의 응답 품질을 높이는 방법 등을 실험적으로 평가했다. 이와 함께 영어로 작성된 프롬프트를 사용해 모델의 다국어 성능을 테스트하며 최적화 방향성을 탐구했다.

2) 적용된 프롬프트 기법

- **Zero-shot CoT** : 문제를 단계적으로 해결하도록 유도하는 방식이다. 모델의 사고 과정을 단계별로 가시화하여 복잡한 문제를 해결할 수 있도록 한다.
- **명확한 규칙 제공 (Plan and Solve)** : 문제 해결 과정을 체계적으로 안내하여 정해진 절차를 따르도록 유도한다.
- **AI 자극 프롬프트** : 모델의 사고 과정을 자극하고 유도하는 방식으로, 보다 창의적이고 유연한 응답을 이끌어낸다.
- **감정적 호소** : 감정적 자극 요소를 삽입하여 모델의 응답 품질을 개선한다. 이는 특정한 맥락에서 더욱 설득력 있는 답변을 유도할 수 있다고 한다.
- **언어 변경** : 다국어로 학습되었다는 Qwen 모델의 특성을 고려하여, 영어 및 중국어로 작성한 프롬프트가 성능에 어떤 영향을 미치는지 평가하였다.

3) 프롬프트 실험 결과

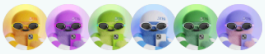
prompt	valid accuracy
Baseline	0.795
plan and solve + zero-shot CoT	0.845
감정적 호소 + zero-shot CoT(영어)	0.855

실험의 목적은 다양한 프롬프트의 valid 정확도를 비교하여 성능을 최대화할 수 있는 템플릿을 선정하는 것이었다. 결과적으로, plan and solve와 zero-shot CoT 방식은 전반적인 성능 향상에 크게 기여했으며, 감정적 호소 및 zero-shot CoT를 영어로 작성한 프롬프트가 성능 향상에 있어 가장 탁월한 효과를 보였다.

3. 최종 결과 제출

증강 + 감정에 호소 ...(1에폭) 감정적 호소 + zero-shot CoT (English)		0.7949 0.7747	2024.11.28 15:05
증강 + RAG 한글 ...(1에폭) plan and solve + zero-shot CoT		0.7765 0.7724	2024.11.28 15:04
증강 + 기존 프롬프트...(1에폭) 기본 프롬프트		0.7926 0.7632	2024.11.28 12:19

Qwen 32B Instruct 모델을 기반으로 HAE-RAE 벤치마크 데이터 증강과 앞서 나온 3가지 프롬프트를 실험에 적용하였다. 이 중 감정적 호소 + zero-shot CoT(영어) 프롬프트를 활용한 조합이 최고 성능을 기록하며 최종 제출에 사용되었다.

1	NLP_05조		0.7977	118	4d
2	NLP_02조		0.7816	132	3d
3	NLP_13조		0.7747	67	4d
4	NLP_06조		0.7747	104	3d
5	NLP_03조		0.7678	93	3d

최종 성적은 mid 기준 5위, fianl 기준 공동 3위로 프로젝트를 마무리할 수 있었다.

4. 자체 평가 의견

A. 잘한 점

초기에 개발 컨벤션을 정하고 YAML 파일로 실험 환경을 체계적으로 관리하며, 각종 문서화 작업과 모듈화, 추가적인 협업 환경 세팅을 통해 팀 내 협업과 커뮤니케이션이 원활하게 이루어졌다. 프로젝트를 시작하며 다양한 접근법(CoT, RAG, 데이터 증강, 대형 모델의 양자화 및 성능 최적화)을 구상하고 이를 모두 시도해보며 실험 및 분석을 수행했다. 또한, Jupyter Notebook의 비효율성을 극복하고자 Streamlit을 도입하여 데이터 분석과 시각화를 효과적으로 수행할 수 있는 데이터 분석 플랫폼을 개발한 점도 잘한 부분으로 꼽을 수 있다.

B. 시도했으나 잘되지 않았던 것들

CoT를 활용했지만, 기대했던 성능 향상을 경험하지 못했다. 마치 만능 해결책처럼 여겨 무조건 적용하면 좋을 것이라 생각한 점이 문제였다. CoT가 주로 수학적 논리나 단계적 사고가 필요한 문제에서 효과적이라는 본질을 간과하고, 단순히 최신 기술이라는 이유만으로 무작정 도입했던 것이다. 결과를 돌아보니 CoT는 모든 문제에 적합하지 않으며, 특히 논리적 사고가 큰 도움이 되지 않는 문제에서는 성능 향상을 기대하기 어렵다는 것을 깨달았다. 사전 분석 없이 기술을 맹신하며 접근했던 점이 가장 아쉬움으로 남는다.

C. 아쉬웠던 점

초기 데이터 관찰과 분석 단계에서 접근 방식이 다소 복잡했던 점이 아쉬움으로 남는다. 초반에는 "외부 지식이 필요한지 여부"를 기준으로 데이터를 간단하게 분류하자는 의견도 있었지만, 보다 복잡하고 다양한 실험이 연속적으로 이루어졌다. 이러한 접근 방식은 직관적으로 이해하기 어려웠고, 결과적으로 가설이 명확하지 않아 실험 결과에 대한 평가와 원인 분석도 쉽지 않았다. 보다 간단하고 명료한 기준으로 접근했다면 결과를 더 효율적으로 도출할 수 있었을 것이라는 아쉬움이 있다.

RAG를 적용하면서 가장 아쉬웠던 점은 벡터 DB의 구성을 충분히 고민하지 못하고 실행에 옮겼다는 것이다. 결과적으로 약 2.2만 개의 임베딩 벡터를 확보했으나, 이는 충분하지 않았던 것으로 판단된다. 또한, 임베딩 모델을 선택할 때도 사전에 철저히 조사하지 않고 단순히 들은 정보를 기반으로 선택했으며, 그 결과 임베딩 품질에 대한 명확한 평가가 이루어지지 않았다. 더불어, retrieve된 문서의 품질과 적합성을 검증하는 과정도 미흡했던 점이 아쉬움으로 남는다.

D. 프로젝트를 통해 배운 점 또는 시사점

대형 모델은 양자화를 하더라도 모델 크기가 클수록 언어 이해 능력이 뛰어나다는 점을 알게 되었다. 또한, 데이터 증강 시 Test 데이터의 분포를 고려하여 높은 품질의 데이터를 증강하는 것이 중요하다는 점을 배웠다. 마지막으로, RAG나 CoT 같은 최신 기술을 도입할 때, 데이터의 특성과 문제의 본질을 면밀히 분석하여 선별적으로 적용해야 성능 향상을 극대화할 수 있다는 교훈을 얻었다.

5. 개인회고

강감찬

좋았던 점 1

가설 및 검증 과정으로 프롬프트에서 필요한 '명확함'을 경험했다. 데이터를 직접 훑어보며, 문제에 정보가 모두 제공된 유형, 수학적/논리적 추론이 필요한 유형 등 분류 기준을 세워 EDA를 수행하였다. EDA 결과, 예측했던 카테고리로 분류가 명확하게 이루어지지 않음을 발견했다. 이를 통해 분류 기준이 모호함을 실험으로 확인할 수 있었고, 외부 지식이 필요한 유형, 필요하지 않은 유형으로 다시 분류하여 데이터 분석을 진행했다. 그 결과, 데이터 분류 작업을 성공적으로 수행할 수 있었다. 가설을 세우고 실험으로 검증하며 가설을 수정하는 과정에서 사람 관점의 직관과 실제 모델이 생각하는 논리의 차이를 경험할 수 있었고, 더 명확하게 모델이 동작하도록 유도할 수 있었다. 프롬프팅에서 필요한 명확한 논리 및 어휘 사용의 중요성을 배울 수 있었다.

좋았던 점 2

데이터 증강이라는 목표를 가지고 세부 실험을 5차례 걸쳐 진행했다. 각 실험 별 평균 4~5회 실험을 진행했고, 실험 결과를 분석하며 실험 설계 시 발견하지 못한 가정의 오류를 발견하기도 했다. 팀 협업 프로젝트에서 기획, 설계, 구현, 분석까지 모두 도맡을 기회가 별로 없었는데, 이번 프로젝트에서는 데이터 증강이라는 세부 작업 내에서 기획, 설계, 구현, 분석까지 전부 경험할 수 있었다.

좋았던 점 3

다른 팀원들과 커뮤니케이션 피드백을 적극적으로 주고 받았다. 이전까지 커뮤니케이션에서 공감과 감정에 기반한 커뮤니케이션 능력은 우수하다는 평가를 많이 받았었다. 하지만 설득이나 주장하는 커뮤니케이션에서는 좋은 평가를 받지 못했다. 흔히 좋은 점에 대한 평가는 받기 쉽지만, 개선해야 할 점, 더군다나 개선 방법과 같이 제시되는 개선점을 피드백을 받는 경우는 매우 드물다. 이번 프로젝트에서는 어떻게 다른 사람들을 설득할 수 있고, 매력적인 주장을 펼칠 수 있는지 커뮤니케이션의 약점인 '자기 생각에 매몰됨'을 토대로 개선점과 함께 피드백을 받을 수 있었다. 최대한 배경 지식을 걷어내고 문장과 문장을 짧게 나눠 그 사이 논리적, 맥락적 빈 틈을 없애라는 피드백은 향후 어떤 직무라도 필요한 기술이라고 생각이 든다.

아쉬운 점 1

데이터 증강 외에 다른 업무를 수행하지 못했다. 프로젝트 기간 내내 데이터 분석과 증강 작업에 몰두한 나머지, 모델 선정 및 경량화, RAG 설계 등 다른 프로세스를 충분히 경험하지 못했다. 특히 LLM 경량화는 많은 기업에서 요구하는 기술인 만큼 반드시 실습해야 한다. 향후 프로젝트 기간 이후에도 여러 방면에서 발견된 개선점을 보완하면서 데이터 증강 뿐만 아닌 다른 업무도 맡고 싶다.

아쉬운 점 2

리더로서 역할이 부족했다. 팀 구성부터 주도적으로 팀원을 영입하고 조율하면서 자연스레 이번 프로젝트 팀 리더를 맡게 되었다. 기술적인 역량이 다른 팀원들에 비해 뛰어나지 않아 프로젝트 진행 시 선두 주자가 아닌, 지원하는 역할을 많이 수행했다. 주로 분위기 조성 및 의견 조율을 맡았고, 회의 시 발언권을 양보하는 축에 속했다. 하지만, 리더보드 마감 직전, 앙상블 대상 선택과 같이 긴박한 상황에서 또한 우선권을 양보했다. 프로젝트 진행이 급박하게 이루어졌기 때문에 팀 목표 설정, 그라운드 룰 세팅 등 팀 단위 협의가 부족했던 것 또한 아쉬움으로 남는다.

단이열

이번 프로젝트를 되돌아보면서 가장 아쉬운 점은 초반 준비가 부족했다는 것이다. 강의 내용을 충분히 숙지하지 못한 상태에서 프로젝트를 시작하다 보니, 초반부터 비효율적인 시간 소모가 많았다. 특히 깃허브 이슈 템플릿과 PR 템플릿을 만드는 데 지나치게 많은 에너지를 쏟아버렸다. 이런 작업이 중요하지 않은 것은 아니지만, 프로젝트의 핵심적인 부분이 아닌 데다 시간이 부족한 상황에서 체력을 너무 소비한 것 같다. 그래도 이런 경험을 통해 업무 우선순위를 정하는 법을 배우고 있다는 점에서는 나름의 의미가 있었다고 생각한다.

스트림릿을 활용하면서 데이터 시각화와 인터페이스 제작이라는 목표는 달성했지만, 돌아보면 데이터를 더 면밀히 분석하며 차분히 접근했어야 하지 않았나 싶다. 다른 팀의 결과물을 보니 1등 팀은 구글 스프레드시트를 활용해 데이터를 정리했는데, 그 방식이 이번 프로젝트의 상황에 더 적합했던 것 같다. 물론 스트림릿이 주는 장점도 있었지만, 효율성과 간편함 측면에서는 스프레드시트가 더 나았을 것이다. 그래도 이런 선택 또한 하나의 경험이라 생각하며, 앞으로는 상황에 맞는 도구를 더 신중히 선택해야겠다는 교훈을 얻었다.

또한, MRC, KMMLU, MMMLU와 같은 다양한 데이터 특성에 대해 깊이 있게 분석하지 못했다는 점이 아쉬움으로 남는다. 데이터에 대한 이해가 부족한 상태에서 작업을 진행하다 보니, 결과적으로 세부적인 디테일이 부족한 결과물이 나왔다. 데이터 분석의 중요성을 다시 한 번 깨닫게 되었고, 이런 부분을 보완하기 위한 노력을 더해야겠다는 생각이 든다.

RAG 관련 작업은 이번 프로젝트에서 가장 뿌듯한 부분이었다. 랭체인을 처음 제대로 활용해 보았고, test final에서 점수가 상승하는 성과를 확인했을 때 정말 기뻤다. 하지만 과정 자체는 매끄럽지 못했다. RAG 개발을 3명이 함께 작업했지만, 업무 분장이 체계적이지 않았고, 하루하루 즉흥적으로 개발을 이어간 느낌이 강하다. 이런 방식은 프로젝트의 효율성을 떨어뜨리기 때문에, 다음번에는 더 철저하게 계획을 세우고 모르는 부분은 즉시 물어보며 진행해야겠다.

RAG의 성능 비교 실험 역시 더 체계적으로 진행했어야 했다고 느낀다. 실험 설계와 실행이 엉성했기 때문에 결과의 신뢰성을 높이는 데 한계가 있었다. 앞으로는 실험 과정 자체에도 더 많은 고민과 시간을 투자해야겠다고 느꼈다.

프로젝트 막판에는 내 일이 바빠지면서 다른 팀원들의 작업을 세심하게 살피지 못한 점도 마음에 남는다. 팀원들과 협업을 잘하려면 내 일만 챙기는 게 아니라, 동료들의 작업에도 관심을 가지고 피드백을 주며 같이 성장하는 자세가 필요하다고 생각한다. 좋은 점은 좋다고 말해주고, 마음에 걸리는 부분도 바로바로 얘기해주는 팀원이 되고 싶다.

아쉬운 점이 많지만, 이번 프로젝트에서 흐지부지 끝나는 일 없이 결과물을 만들어냈다는 점은 큰 성취로 느껴진다. 매일매일 팀원들과 커뮤니케이션을 이어가며 어떻게든 맡은 일을 해내려고 노력했다. 저번 프로젝트와 비교했을 때, 뚜렷한 결과물을 냈다는 것만으로도 성장했다고 느낀다. 앞으로는 이런 경험을 밑바탕 삼아 더 좋은 성과를 내고 싶다.

안혜준

이번 프로젝트는 LLM을 활용해 수능 문제를 해결하는 모델을 개발하는 대회로, 본격적으로 LLM을 사용해 작업한 첫 프로젝트였던 만큼 매우 즐겁게 참여할 수 있었다. 새로운 팀원들과 함께 진행된 프로젝트였던 만큼 각자의 스타일을 조율하며 원활한 협업을 위해 많은 고민을 했다. 프로젝트 초반에는 팀 내 규칙을 명확히 정해 개발과 실험이 매끄럽게 이루어질 수 있도록 준비했다.

개발적인 측면에서는 코드 스타일을 통일하기 위해 black, isort, flake8과 같은 도구를 도입해 linter를 활용한 코드 포맷팅을 강제했고, GitHub Actions를 활용해 PR 단계에서 자동으로 lint 검사를 실행할 수 있도록 설정했다. Git 기반 협업을 원활히 하기 위해 gitcommit 템플릿과 이슈/PR 템플릿을 도입했으며, Github Flow 전략을 채택해 브랜치를 효율적으로 관리했다. 이러한 체계적인 준비 덕분에 팀원들이 빠르게 적응하며 효과적으로 협업할 수 있었다.

본인이 이번 프로젝트에서 중점적으로 진행했던 부분은 Chain of Thought(CoT) 방식을 활용해 문제를 해결하는 접근이었다. CoT 방식은 모델이 단계적인 추론 과정을 거쳐 정답을 도출하도록 유도하는 접근법으로, 특히 어려운 문제를 단계적으로 접근함으로써 더 쉽게 해결할 수 있을 것이라는 기대 아래 여러 방식으로 실험을 진행했다. 그러나 결과적으로 CoT 방식은 기대에 미치지 못한 성과를 보였다.

가장 큰 원인은 데이터의 특성이었다. CoT 방식은 수학 문제처럼 논리적 단계가 명확히 요구되는 작업에서 효과를 발휘하는 반면, 국어나 사회 문제는 단계적인 추론보다 직관적인 답변이 더 적합한 경우가 많았다. 예를 들어, 주어진 텍스트에서 단순히 지문과 선택지를 비교해 답을 찾을 수 있는 문제에서는 CoT 방식이 불필요한 과정을 추가하며 오히려 성능을 저하시켰다.

또 다른 문제는 모델의 파라미터 한계였다. 실험에 사용된 google/gemma-2-2b-it 모델은 CoT 방식의 복잡성을 충분히 처리하기엔 용량이 부족했던 것으로 보였다. 모델이 추론 과정을 진행하다가도 논리적 일관성을 잃거나, 예상치 못한 외국어 응답을 생성하는 등 비정상적인 결과를 보이는 사례가 발생했다.

모델 응답 형식의 차이 역시 성능 저하의 주요 원인 중 하나였다. 기존의 간단히 정답 번호를 출력하는 방식과 비교했을 때, JSON 형식의 응답은 복잡도가 증가해 모델의 학습과 추론에 더 큰 부담을 주었다. JSON 형식으로 파인튜닝을 진행했음에도, 테스트 데이터 중 약 30개의 문제에서 제대로 된 구조의 JSON 생성에 실패하는 사례가 확인되었다. 이러한 실패는 파인튜닝 이후에도 성능 향상을 제한하는 요소로 작용했다.

이번 실험에서는 CoT 방식의 적용이 데이터 특성에 더 적합하게 이루어졌다면 결과가 더 좋았을 것이라는 아쉬움이 남는다. CoT 방식이 효과적인 문제 유형(예: 수학, 논리)을 사전에 분석하고, 적합한 데이터셋에만 선택적으로 적용했다면 불필요한 복잡성을 줄일 수 있었을 것이다. 또한, JSON 형식 대신 reasoning과 answer를 단순한 텍스트 형식으로 생성하도록 설계했다면 모델의 부담을 덜어 성능 저하를 방지할 수 있었을 것이다. 마지막으로, 대형 모델을 활용했다면 CoT 방식의 잠재력을 더 효과적으로 발휘할 수 있었을 것으로 보인다. 이번 실험에서 이러한 부분들을 충분히 고려하지 못한 점은 아쉽지만, 이를 교훈 삼아 앞으로 더 나은 접근 방식을 설계할 수 있을 것이다.

유선우

학습 목표 달성을 위한 노력

팀의 학습 목표

- **다양한 시도**: 팀원 각자가 새로운 아이디어와 기술을 적극적으로 시도하여 프로젝트의 다양성을 높였습니다.
- **모두의 성장**: 개인의 역량 강화뿐만 아니라 팀 전체의 성장에 기여하는 것을 목표로 삼았습니다.

개인 학습 측면

- **코드 모듈화 수준 향상**: 베이스라인 코드를 체계적으로 모듈화하여 코드의 재사용성과 유지보수성을 크게 높였습니다.
- **다양한 모델 탐색 및 실험**: Hugging Face의 다양한 모델을 탐색하고 실험하여 최적의 모델을 선정하기 위해 노력했습니다.
- **메모리 최적화 기법 연구**: 양자화, Unsloth, LoRA, Gradient Accumulation 등 다양한 메모리 최적화 기법을 학습하고 적용하여 대용량 모델의 학습을 가능하게 했습니다.

공동 학습 측면

- **지식 공유**: 팀원들과 정기적으로 모여 각자의 조사와 연구 내용을 공유하고 토론함으로써 팀의 지식을 향상시켰습니다.
- **코드 리뷰 및 협업**: 이슈를 기반으로 PR을 작성하고 코드 리뷰를 진행하여 코드의 효율성과 안정성을 높였습니다.

모델 개선 방식

사용한 지식과 기술

- **모델 탐색 및 실험**: 다양한 사전 학습된 모델을 비교 분석하여 프로젝트에 가장 적합한 모델을 선정했습니다.
- **양자화를 통한 대형 모델 학습**: 양자화 기법을 적용하여 메모리 한계로 학습이 어려웠던 대형 모델을 효과적으로 학습할 수 있었습니다.
- **하이퍼파라미터 최적화**: 여러 실험을 통해 최적의 하이퍼파라미터 조합을 찾아 모델의 성능을 향상시켰습니다.

달성한 지점과 얻은 깨달음

- **메모리 제약 극복**: 메모리 최적화 기법을 적용하여 GPU 메모리 한계로 로드할 수 없었던 대형 모델을 성공적으로 활용할 수 있었습니다.
- **성능 향상**: 다양한 모델 실험과 하이퍼파라미터 튜닝을 통해 모델의 정확도를 크게 높였습니다.
- **협업의 중요성 인식**: 팀원들과의 지식 공유와 협업을 통해 개인의 한계를 넘어설 수 있음을 깨달았습니다.

전과 비교하여 새롭게 시도한 변화와 효과

- **한 단계 발전한 코드 모듈화**: 코드 구조를 체계적으로 설계하고 모듈화하여 유지보수성과 확장성을 크게 향상시켰습니다. 이를 통해 팀원들과의 협업이 더욱 원활해지고 코드의 품질이 높아졌습니다.
- **메모리 최적화 기법의 적극적 도입**: 다양한 메모리 최적화 기술을 적용하여 모델의 크기를 유지하면서도 메모리 사용량을 감소시켰습니다. 이를 통해 더 복잡하고 성능이 뛰어난 모델을 활용할 수 있었습니다.

마주한 한계와 아쉬운 점

- **학습 시간의 증가**: 메모리 최적화를 위해 적용한 일부 기법이 학습 속도를 저하시키는 부작용이 있었습니다.
- **기술적 이해의 부족**: 새로운 기술을 적용하면서 일부 원리에 대한 깊은 이해가 부족하여 아쉬움이 남았습니다.
- **오류 수정에 많은 시간 소요**: 예상치 못한 오류로 인해 수정에 많은 시간이 소요되었습니다.

다음 프로젝트에서의 시도점

- **효율적인 학습 기법 연구**: 메모리 최적화와 함께 학습 속도를 개선할 수 있는 방법을 더 탐구하고자 합니다.
- **이론 심화 학습**: 적용한 기술들의 이론적 배경과 원리를 깊이 있게 공부하여 더 안정적이고 효율적인 코드를 작성하고자 합니다.

- **지속적인 협업 강화**: 팀원들과의 협업을 더욱 강화하여 지식 공유와 공동 성장을 도모할 것입니다.

유채은

이번 프로젝트에서 제 개인적인 목표는 성능 수치를 나침반 삼아 새로운 기술을 학습하는 것이었습니다. 이전 ODQA 프로젝트에서 다뤘던 dense embedding을 심화 학습하고 싶었고, 사용해 본 적 없던 Streamlit을 활용해 시각화를 진행해보고 싶다는 하위 목표를 그리고 있었는데, 운 좋게 이 두 가지 모두 해당 프로젝트를 통해 경험할 수 있었습니다.

새 팀과 함께한 첫 번째 프로젝트였기에, 프로젝트 기간 초반에는 서로 익숙한 방식들을 공유하고 팀 차원에서 이를 조율하는 과정이 필요했습니다. 자신이 보다 낫다고 생각하는 협업 방식을 근거와 함께 제시하고, 타인의 의견이 합리적이라면 자신의 익숙한 방식을 내려놓는 과정을 통해 새로운 팀의 그라운드룰을 설정할 수 있었고 이를 바탕으로 저희만의 팀워크를 형성할 수 있던 것이 원활한 협업의 기반이 되었습니다.

프로젝트 기간 초반 제 역할은 데이터 분석 및 전처리였습니다. 분석 결과와 분석 방식을 활용해 Streamlit에 시각화를 진행하여 데이터의 흐름을 직관적으로 파악할 수 있었고, 문서 길이 분포 등을 확인하며 새로 도입할 데이터가 기존 데이터와 유사한 분포를 가지는지 검증할 수 있었습니다. Streamlit을 통해 데이터 사용 여부 등의 의사결정과 수정 작업을 훨씬 효율적으로 진행할 수 있어 시각화의 중요성을 체감할 수 있었습니다.

이후에는, RAG 구현에 참여하여 모델이 수능 문제 풀이에 적합한 문서를 힌트로 활용할 수 있도록 하였습니다. RAG를 적용하는 과정에서는 예상과 달리 점수 향상이 크지 않아 다각적인 분석과 가설 검증을 통해 원인을 탐구했습니다.

1) 우리의 Vector DB에 매력적인 힌트 문서들이 없다. -> 보다 수능과 관련된 문서 추가 (요점정리 pdf 등)
2) query를 지문만으로 설정하였을 때, retrieve된 힌트 문서가 문제를 푸는데 도움이 될 확률이 낮다. -> query 생성 시 지문 + 질문 + 선택지를 전부 활용하도록 수정정
3) query 생성 시 지문 + 질문 + 선택지 간의 밸런스가 맞지 않다. -> 지문을 요약하여 지문, 질문, 선택지의 비중을 최대한 동등하게 유지

등 문제 상황이 발생할 때마다 그에 대한 가설 세우고 가설이 맞는지 수치적 검증을 진행하였습니다. retrieval의 평가에 llm as judge를 활용한 것이 성능을 정량화하기에 효율적이었으며, 이열 캠퍼의 조언을 들어 제작한 retrieval rubic을 바탕으로 gpt 4o에게 평가를 맡겼을 때 2, 3번 가설과 어느정도 부합하는 경향성을 확인할 수 있었습니다.

이 과정에서 팀원들과 긴밀히 협력하며 문제를 파악하는 연습을 많이 할 수 있었습니다. 또한 제한된 시간 내에 좋은 실험을 진행하기 위해 실험 방식에 취사선택이 요구되었는데, 제가 옳다고 생각하는 방식을 내세울 때 합리적인 근거를 함께 제시할 수 있는 커뮤니케이션 스킬에 대해 공부할 수 있었습니다. 최종적으로 중간 점수에서는 성능향상을 보이지 못해 RAG 구성이 최종 모델에 채택되지 않았지만, 후에 final 점수가 공개되었을 때, gpt 4o를 채점관으로 활용한 결과와 유사하게 지문, 질문, 선택지를 모두 활용한 query에서 점수 향상이 있었음을 확인하였고, 수치를 길잡이로 삼고 나아간 방향성이 틀리지 않았음을 확인해 기뻤습니다.
이 모든 과정을 되돌아보면 시행착오에 소요된 시간이 가장 아쉽습니다. 이러한 경험을 통해 체계적인 자료조사와 검증의 중요성을 배웠으며, 이를 바탕으로 다음 프로젝트에서는 더 효율적인 접근을 위해 자료조사에 더 많은 시간을 투자할 계획입니다.

이채호

이번 프로젝트에서는 특히나 경험이 많은 새로운 팀원들에게 최대한 배우면서 기여하는 태도를 적극적으로 실천하려고 노력했다. 이러한 태도 덕분에 비록 길지 않은 프로젝트 기간이었지만 많은 성장을 이루어낼 수 있었다. 협업을 위한 환경 설정부터 깃 문서 작업, 브랜치 관리 등 새로운 일들이 많았지만, 이내 익숙해지며 일처리를 할 수 있게 되는 스스로의 모습이 만족스러웠다. 지속 가능한 성장을 위해 어떤 마인드를 가져야 하는지 다시 한번 되새기게 되는 경험을 할 수 있었다.

이번 프로젝트에서는 논문을 읽고 활용하는 과정을 통해 많은 것을 배울 수 있었다. 멘토님께서 멘토링 중에 해주신 조언을 기반으로 논문에서 배운 정보를 적극적으로 프로젝트에 적용하려고 노력했다. 하지만 논문을 읽고 실제로 적용하는 경험이 부족해 멘토님께 어디서부터 정보를 찾아봐야 할지 추가적인 조언을 구하기도 했다. 멘토님은 현재 데이터셋의 소스 중 KMMLU가 있으니 이에 관련된 논문을 찾아보는 것을 제안해 주셨다. 이를 계기로 KMMLU 벤치마크 논문을 읽으며 데이터셋의 특성을 더 잘 이해할 수 있었고, 논문에서 소개된 여러 벤치마크 데이터셋들을 선별해 훈련 데이터에 취합했다. 그 결과, 베이스라인 코드 학습 후 모델의 Accuracy를 기존 0.3986에서 0.4309로 대폭 향상시키는 성과를 거둘 수 있어 매우 뿌듯했다. 이러한 경험은 논문이 단순한 지식의 저장소가 아니라, 실질적인 문제 해결을 위한 강력한 도구임을 깨닫게 해주었다.

그러나 이번 프로젝트에서 아쉬운 점도 적지 않았다. 첫 번째로, 논문에서 데이터를 발췌하는 수준에 그쳤을 뿐, 논문에서 배운 내용을 충분히 응용하지 못했다는 점이다. 논문에서 소개된 지식을 깊이 이해하고 이를 실제 문제에 어떻게 활용할 수 있는지에 대한 통찰을 얻는 데 한계가 있었다. 이를 개선하기 위해서는 평소에 논문을 꾸준히 읽고, 이해한 내용을 직접 실습하며 적용해 보는 과정을 거치는게 꼭 필요하다는 것을 깨달았다. 또한, 모델에 대한 수학적 이해도와 구현 능력을 함께 키워야 이러한 한계를 극복할 수 있을 것이라는 교훈을 얻었다.

또 다른 아쉬움은, RAFT를 구현하겠다는 목표를 세웠지만, 진행 중이던 다른 작업에 밀려 이를 다른 팀원에게 부탁하게 된 점이다. 목표를 스스로 달성하지 못했다는 점에서 아쉬움이 컸지만, 팀원이 적극적으로 태스크를 진행해준 덕분에 프로젝트가 무리 없이 진행될 수 있었다. 이 과정에서 책임감을 느끼며, 다음 프로젝트 때에는 더 적극적으로 작업의 방향성을 제시하고 모델 성능 개선에 주도적인 역할을 하고 싶다는 욕심이 생기는 계기가 됐다. 앞으로는 시간 관리와 우선순위 설정을 더 철저히 하여 목표를 완수하고, 팀의 기여도를 높이는 데 집중하고자 한다.

하지만 이번 프로젝트를 통해 얻은 긍정적인 점도 많았다. 프로젝트에 도움이 되는 정보를 어디서 어떻게 찾고 응용해야 하는지 조금이나마 감을 잡게 되었고, 이를 통해 스스로 학습하고 발전할 수 있는 방법을 터득하게 된 것 같다. 논문에서 얻은 정보로 구체적인 성과를 만들어낸 경험은 앞으로의 프로젝트에서도 큰 자신감이 될 것이다. 또한, 협업과 소통의 중요성을 체감하며, 팀원들과 함께 성장할 수 있는 프로젝트 문화의 가치를 다시금 깨달았다.